

ATIVIDADE 11**Análise Estática e Refatoração no VS Code****Aluno**

• Beto da Silva — RA 12525178737

1. Código original (ProcessadorPagamento.java)

```
package src;

import java.util.*;

public class ProcessadorPagamento {

    public void processar(String tipo, double valor, String cartao) {

        if (tipo != null && !tipo.isEmpty()) {
            if (valor > 0) {
                if (cartao != null && cartao.length() == 16) {

                    if (tipo.equals("credito")) {
                        System.out.println("Processando crédito de R$ " + valor);
                        double taxa = valor * 0.05;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("debito")) {
                        System.out.println("Processando débito de R$ " + valor);
                        double taxa = valor * 0.02;
                        double total = valor + taxa;
                        System.out.println("Total com taxa: R$ " + total);
                    } else if (tipo.equals("pix")) {
                        System.out.println("Processando PIX de R$ " + valor);
                    } else {
                        System.out.println("Tipo inválido");
                    }
                }
            }
        } else {
            System.out.println("Cartão inválido");
        }
    } else {
        System.out.println("Valor inválido");
    }
} else {
    System.out.println("Tipo inválido");
}
}

public boolean validarSenha(String senha) {
```

```

String senhaCorreta = "admin123";
if (senha == senhaCorreta) {
    return true;
}

return false;
}

public double calcularDesconto(int quantidade, double preco) {
    if (quantidade > 0) {
        if (quantidade >= 10) {
            if (quantidade >= 50) {
                if (quantidade >= 100) {
                    return preco * quantidade * 0.7;
                } else {
                    return preco * quantidade * 0.8;
                }
            } else {
                return preco * quantidade * 0.9;
            }
        } else {
            return preco * quantidade;
        }
    } else {
        return 0;
    }
}
}

```

2. Tabela de Registro de Problemas

Problemas identificados executando de verdade Checkstyle 9.3, SpotBugs 4.10.4 e Semgrep 1.179.0 (com duas regras locais) sobre o código original.

ID	Ferramenta	Linha	Descrição do Problema	Tipo	Severidade
1	Checkstyle	3	Import wildcard java.util.* reduz clareza e controle de dependências	Estilo/Manutenibilidade	Baixa
2	Checkstyle	5	Classe sem documentação Javadoc	Estilo/Manutenibilidade	Baixa
3	Checkstyle	7	Método processar sem documentação Javadoc	Estilo/Manutenibilidade	Baixa
4	Checkstyle	40	validarSenha sem documentação Javadoc	Estilo/Manutenibilidade	Baixa
5	Checkstyle	49	calcularDesconto sem	Estilo/Manutenibilidade	Baixa

			documentação Javadoc		
6	SpotBugs	42	Comparação de senha String com == verifica identidade, causando autenticação incorreta	Bug	Alta
7	Semgrep	41	Senha fixa hardcoded em senhaCorreta	Segurança	Alta
8	Semgrep	42	Comparação de String usando ==	Bug	Alta

O SpotBugs registrou o bug do == uma única vez; o Semgrep registrou a senha fixa e o == separadamente, para que cada correção fique rastreável.

3. Técnicas de refatoração aplicadas

Técnica	O que foi feito
Early Return	processar() valida tipo/valor/cartão com retorno antecipado, sem aninhar ifs em cascata
Extract Method	processarTipo() e processarComTaxa() extraem a lógica antes repetida nos blocos de crédito/débito
Substituir == por comparação segura	validarSenha() usa MessageDigest.isEqual() (resistente a timing attack) em vez de == entre Strings
Remover senha hardcoded	Credencial (hash + salt) fornecida externamente via criarComCredencial(saltBase64, hashBase64); validação por PBKDF2-HMAC-SHA256, 600.000 iterações, conforme OWASP
Extrair Constantes	TAXA_CREDITO/DEBITO, TAMANHO_CARTAO, LIMITE_DESCONTO_10/20/30, FATOR_DESCONTO_10/20/30, ITERACOES_PBKDF2 e outras substituem os números mágicos
Adicionar Javadoc	Javadoc com @param/@return/@throws na classe e nos métodos públicos
Corrigir indentação	Padronizado em 4 espaços por nível (perfil usado na comparação principal do Checkstyle)
Imports explícitos	import java.util.*; substituído por imports específicos (MessageDigest, SecretKeyFactory, PBEKeySpec, Base64, Arrays etc.)

4. Código refatorado (ProcessadorPagamento.java corrigido)

```

package src;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.security.spec.InvalidKeySpecException;
import java.util.Arrays;
import java.util.Base64;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;

/** Processa pagamentos e descontos e verifica uma credencial fornecida
externamente. */
public class ProcessadorPagamento {

    private static final double TAXA_CREDITO = 0.05;
    private static final double TAXA_DEBITO = 0.02;
    private static final int TAMANHO_CARTAO = 16;
    private static final int LIMITE_DESCONTO_10 = 10;
    private static final int LIMITE_DESCONTO_20 = 50;
    private static final int LIMITE_DESCONTO_30 = 100;
    private static final double FATOR_DESCONTO_10 = 0.9;
    private static final double FATOR_DESCONTO_20 = 0.8;
    private static final double FATOR_DESCONTO_30 = 0.7;
    private static final int ITERACOES_PBKDF2 = 600_000;
    private static final int TAMANHO_HASH_BITS = 256;
    private static final int TAMANHO_HASH_BYTES = 32;
    private static final int MINIMO_SALT_BYTES = 16;
    private static final String ALGORITMO_SENHA =
"PBKDF2WithHmacSHA256";

    private final byte[] salt;
    private final byte[] hashEsperado;

    private ProcessadorPagamento(byte[] salt, byte[] hashEsperado) {
        this.salt = salt.clone();
        this.hashEsperado = hashEsperado.clone();
    }

    /**
     * Cria o processador a partir de hash e salt persistidos pelo cadastro.
     *
     * @param saltBase64 salt aleatorio de pelo menos 16 bytes, codificado
em Base64
     * @param hashBase64 hash de 32 bytes, codificado em Base64
     * @return processador configurado
     * @throws IllegalArgumentException se os dados estiverem ausentes,
malformados ou invalidos
     */
    public static ProcessadorPagamento criarComCredencial(String
saltBase64, String hashBase64) {
        if (saltBase64 == null || hashBase64 == null) {
            throw new IllegalArgumentException("Salt e hash devem ser
configurados");
        }
        byte[] saltConfigurado;
        byte[] hashConfigurado;
        try {
            saltConfigurado = Base64.getDecoder().decode(saltBase64);
            hashConfigurado = Base64.getDecoder().decode(hashBase64);

```

```

        } catch (IllegalArgumentException erro) {
            throw new IllegalArgumentException("Credencial Base64 invalida",
erro);
        }
        if (saltConfigurado.length < MINIMO_SALT_BYTES
            || hashConfigurado.length != TAMANHO_HASH_BYTES) {
            throw new IllegalArgumentException("Tamanho de salt ou hash
invalido");
        }
        return new ProcessadorPagamento(saltConfigurado,
hashConfigurado);
    }

    /**
     * Processa um pagamento, preservando as mensagens e as regras do
exercicio original.
     *
     * <p>Por compatibilidade com o enunciado, o cartao de 16 caracteres
tambem e exigido
     * para PIX. Esta e uma limitacao didatica, nao uma regra de um sistema
real de PIX.
     *
     * @param tipo tipo de pagamento: credito, debito ou pix
     * @param valor valor da transacao
     * @param cartao identificador de cartao com 16 caracteres
     */
    public void processar(String tipo, double valor, String cartao) {
        if (tipo == null || tipo.isEmpty()) {
            System.out.println("Tipo inválido");
            return;
        }
        if (!(valor > 0)) {
            System.out.println("Valor inválido");
            return;
        }
        if (cartao == null || cartao.length() != TAMANHO_CARTAO) {
            System.out.println("Cartão inválido");
            return;
        }
        processarTipo(tipo, valor);
    }

    private void processarTipo(String tipo, double valor) {
        switch (tipo) {
            case "credito":
                processarComTaxa("crédito", valor, TAXA_CREDITO);
                break;
            case "debito":
                processarComTaxa("débito", valor, TAXA_DEBITO);
                break;
            case "pix":
                System.out.println("Processando PIX de R$ " + valor);
                break;
            default:
                System.out.println("Tipo inválido");
                break;
        }
    }
}

```

```

private void processarComTaxa(String descricao, double valor, double
taxa) {
    System.out.println("Processando " + descricao + " de R$ " + valor);
    double total = valor + valor * taxa;
    System.out.println("Total com taxa: R$ " + total);
}

/**
 * Verifica a senha por derivacao PBKDF2 e comparacao dos bytes do
hash.
 *
 * @param senha senha informada pelo chamador
 * @return true quando a credencial coincide; false para senha nula ou
incorreta
 * @throws IllegalStateException se o algoritmo de derivacao nao estiver
disponivel
 */
public boolean validarSenha(String senha) {
    if (senha == null) {
        return false;
    }
    char[] caracteres = senha.toCharArray();
    PBEKeySpec especificacao = new PBEKeySpec(
        caracteres, salt, ITERACOES_PBKDF2,
TAMANHO_HASH_BITS);
    try {
        SecretKeyFactory fabrica =
SecretKeyFactory.getInstance(ALGORITMO_SENHA);
        byte[] hashInformado =
fabrica.generateSecret(especificacao).getEncoded();
        try {
            return MessageDigest.isEqual(hashEsperado, hashInformado);
        } finally {
            Arrays.fill(hashInformado, (byte) 0);
        }
    } catch (NoSuchAlgorithmException | InvalidKeySpecException erro) {
        throw new IllegalStateException("Falha ao verificar credencial",
erro);
    } finally {
        especificacao.clearPassword();
        Arrays.fill(caracteres, '\0');
    }
}

/**
 * Calcula o total com desconto progressivo de acordo com a quantidade.
 *
 * @param quantidade quantidade de itens
 * @param preco preco unitario
 * @return zero para quantidade nao positiva; total com o desconto
aplicavel nos demais casos
 */
public double calcularDesconto(int quantidade, double preco) {
    if (quantidade <= 0) {
        return 0;
    }
    if (quantidade >= LIMITE_DESCONTO_30) {
        return preco * quantidade * FATOR_DESCONTO_30;
    }
}

```

```

    if (quantidade >= LIMITE_DESCONTO_20) {
        return preco * quantidade * FATOR_DESCONTO_20;
    }
    if (quantidade >= LIMITE_DESCONTO_10) {
        return preco * quantidade * FATOR_DESCONTO_10;
    }
    return preco * quantidade;
}
}

```

5. Comparação Antes/Depois

Métrica	Antes	Depois	Melhoria
Total de problemas (Checkstyle, 4 espaços)	5	0	100% menos
Total de problemas (SpotBugs)	1	0	100% menos
Total de problemas (Semgrep)	2	0	100% menos
Complexidade ciclomática máxima	9	6	33% menos
Profundidade máxima de if aninhado	3	0	100% menos
Comparação de senha String com ==	1	0	100% menos
Senha fixa no código (Semgrep)	1	0	100% menos
Números mágicos (Checkstyle dedicado)	9	0	100% menos

Regra de contagem: o Checkstyle principal foi medido com as regras Google e indentação ajustada para 4 espaços, como pede a atividade. O padrão de 2 espaços do Google foi executado à parte e registrado em XML; como a exigência de indentação conflita com esse perfil padrão, a contagem padrão não é usada na comparação principal.

6. Validação funcional

APROVADO: 230 verificações. Foram comparadas 168 combinações de pagamentos e 40 combinações de descontos com a versão original, além de 10 limites de cartão, quatro entradas de senha e cinco configurações inválidas. Nenhuma senha de teste foi gravada em arquivo.

7. Decisão de segurança

A refatoração não substitui a senha fixa por outro segredo codificado. O cadastro deve fornecer hash PBKDF2-HMAC-SHA256 e salt aleatório em Base64. A verificação usa 600.000 iterações, 256 bits e comparação resistente a diferenças de tempo (timing attack), conforme a OWASP Password Storage Cheat Sheet. Em produção, o cadastro e o armazenamento precisam se integrar a um serviço de credenciais seguro.

8. Decisão de compatibilidade

O código conserva a regra original de exigir cartão com 16 caracteres inclusive para PIX, para permitir a comparação comportamento a comportamento. O Javadoc identifica isso como limitação didática. O

dinheiro continua sendo double porque essa é a interface proposta no exercício.

9. Ferramentas e evidências

Checkstyle 9.3 (mesmo jar da extensão VS Code), SpotBugs 4.10.4 e Semgrep 1.179.0 com duas regras locais incluídas no projeto. Os resultados completos (XML/JSON, logs, código original compilado e o refatorado) estão nas pastas do projeto.

10. Pendências de entrega

- Print do painel Problems (Ctrl+Shift+M) mostrando os problemas identificados ANTES da refatoração
- Print do painel Problems mostrando os problemas (idealmente zerados) DEPOIS da refatoração

Esses dois prints precisam ser capturados rodando o VS Code com as extensões instaladas (Checkstyle, SpotBugs, Semgrep) — não é possível gerá-los sem executar o ambiente real localmente.